An Introduction to Applied Bayesian Statistics with Stan

2026-08-17

Overview

- **Goals**

  Provide the foundation and orientation so that you can

  - Use Stan (or **brms**) in your work
  - Understand and enjoy StanCon talks!

- **Outline**
  - About Stan ✓
  - Bayesian models and workflow ✓
  - Model specification in Stan ←
  - Multilevel model specification in **brms**
  - HMC and multi-level models

- **Slides** stancon_2026_stan_intro

The Stan language

The Stan Probabilistic Programming Language

- The Stan language is a Turing-complete imperative programming language **for specifying differentiable log densities** and **posterior predictive inferences**.
- Syntax
  - Named program blocks
  - Functions (including recursion)
  - Explicit variable types (like Java/C++, not like Python, R)
  - Reassignable local variables and scoping
  - Control flow: `if` statements - dynamic branch points (more than a DAG!)
- Mathematical operations
  - Stan language includes a very large set of probability distributions and mathematical functions from Stan's math library
  - Efficient, vectorized (almost always)

Stan Program File

A Stan program consists of one or more named program blocks, strictly ordered

```
functions {
  // declare, define functions
} data {
  // declare input data
} transformed data {
   // transform inputs, define program data
} parameters {
   // declare (continuous) parameters
} transformed parameters {
   // define derived parameters
} model {
   // compute the log joint distribution
} generated quantities {
   // define quantities of interest
}
```

Basic Program Blocks

- `data` (execute once)
  - *content*: declare data types, sizes, and constraints
  - *execute*: read from data source, validate constraints
- `parameters` (initialize at startup, execute every log prob eval)
  - *content*: declare parameter types, sizes, and constraints
  - *execute*: transform to constrained, Jacobian
- `model` (execute every log prob eval)
  - *content*: statements defining posterior density
  - *execute*: execute statements

Derived Variable Blocks

- `transformed data` (execute once after data)
  - *content*: declare and define transformed data variables
  - *execute*: execute definition statements, validate constraints
- `transformed parameters` (execute every log prob eval)
  - *content*: declare and define transformed parameter vars
  - *execute*: execute definition statements, validate constraints
- `generated quantities` (execute once per draw, `double` type)
  - *content*: declare and define generated quantity variables; includes pseudo-random number generators

    (for posterior predictions, event probabilities, decision making)
  - *execute*: execute definition statements, validate constraints

Reading and Transforming Data

- Only done once (per chain)
  - computation is inexpensive
- Read data
  - read data variables from memory or file stream
  - validate data
- Generate transformed data
  - execute transformed data statements
  - validate variable constraints when done

Constraining and Unconstraining Parameters

- Inference algorithms operate on parameter values in unconstrained $\mathbb{R}^n$
  - algorithms can move freely in any direction
  - no boundaries to cross or constraints to enforce
- Stan programs declare parameters on their **constrained scale**;

  this matches the model semantics.
  - `real<lower=0>` for a positive parameter
  - `simplex[K]` for a probability vector
  - `corr_matrix[K]` for a correlation matrix
- Stan automatically transforms between the two representations
  - **unconstrain** parameter values for use by the inference algorithm
  - **constrain** parameter values for use in the Stan program
- The user specifies the constraints; Stan handles the transformations

Evaluating the Joint Log Density

- **Given** current values of all model parameters on the unconstrained scale
- Stan computes the joint log density and its gradient:
  - transform parameters to satisfy their declared constraints
  - execute the `transformed parameters` block
  - execute the `model` block to accumulate the joint log density
  - account for parameter transformations as needed
  - automatically compute gradients with respect to the unconstrained parameters
- The inference algorithm uses the log density and gradients to explore the posterior
- Producing one posterior draw may require **many** such evaluations
  - HMC's leapfrog algorithm takes up to $2^{\text{max\_treedepth}}$ evaluations

Evaluating Generated Quantities

- **Given** parameter values
- Once per iteration (not once per leapfrog step)
- May involve (pseudo) random-number generation
  - Executed generated quantity statements
  - Validate values satisfy constraints
- Typically used for
  - Event probability estimation
  - Predictive posterior estimation
- Efficient because evaluated with `double` types (no autodiff)

Variable and Expression Types

Variables and expressions are **strongly, statically typed**.

- **Primitive**: `int`, `real`
- **Matrix**: `matrix[M,N]`, `vector[M]`, `row_vector[N]`
- **Bounded**: primitive or matrix, with `<lower=L>`, `<upper=U>`, `<lower=L,upper=U>`
- **Constrained Vectors**: `simplex[K]`, `ordered[N]`, `pos_ordered[N]`, `sum_to_zero[K]`, `positive_ordered[N]`, `unit_length[N]`
- **Constrained Matrices**: `cov_matrix[K]`, `corr_matrix[K]`, `cholesky_factor_cov[M,N]`, `cholesky_factor_corr[K]`, `row_stochastic_matrix[M, N]`, `column_stochastic_matrix[M, N]`
- **Arrays:** of any type (and dimensionality) `array[N] T`

Integers and Reals

- Different types (conflated in BUGS, JAGS, and R)
- Distributions and assignments care
- Integers may be assigned to reals but not vice-versa
- Reals have not-a-number, and positive and negative infinity
- Integers single-precision up to +/- 2 billion
- Integer division rounds (Stan provides warning)
- Real arithmetic is inexact and reals should not be (usually) compared with ==

Arrays, Vectors, and Matrices

- Stan separates arrays, matrices, vectors, row vectors
- Which to use?
- Arrays allow most efficient access (no copying)
- Arrays stored first-index major (i.e., 2D are row major)
- Vectors and matrices required for matrix and linear algebra functions
- Matrices stored column-major (memory locality matters)
- Are not assignable to each other, but there are conversion functions, see Stan Functions Reference chapter Mixed Operations

Logical Operators

Op.	*Prec.*	*Assoc.*	*Placement*	*Description*
`||`	9	left	binary infix	logical or
`&&`	8	left	binary infix	logical and
`==`	7	left	binary infix	equality
`!=`	7	left	binary infix	inequality
`<`	6	left	binary infix	less than
`<=`	6	left	binary infix	less than or equal
`>`	6	left	binary infix	greater than
`>=`	6	left	binary infix	greater than or equal

Precedence: order of evaluation, e.g. `1 + 2 * 3` evaluates to 7 (not 6).

Logical operators are lower precedence than arithmetic and matrix operators.

Arithmetic and Matrix Operators

Op.	*Prec.*	*Assoc.*	*Placement*	*Description*
`+`	5	left	binary infix	addition
`-`	5	left	binary infix	subtraction
`*`	4	left	binary infix	multiplication
`/`	4	left	binary infix	(right) division
`\`	3	left	binary infix	left division
`.*`	2	left	binary infix	elementwise multiplication
`./`	2	left	binary infix	elementwise division
`!`	1	n/a	unary prefix	logical negation
`-`	1	n/a	unary prefix	negation
`+`	1	n/a	unary prefix	promotion (no-op in Stan)
`^`	2	right	binary infix	exponentiation
`'`	0	n/a	unary postfix	transposition
`()`	0	n/a	prefix, wrap	function application
`[]`	0	left	prefix, wrap	array, matrix indexing

Assignment Operators

Op.	*Description*
`=`	assignment
`+=`	compound add and assign
`-=`	compound subtract and assign
`*=`	compound multiply and assign
`/=`	compound divide and assign
`.*=`	compound elementwise multiply and assign
`./=`	compound elementwise divide and assign

- these work with all relevant matrix types
  - e.g., `matrix *= matrix;`

Built-in functions

- All built-in **C++ functions and operators**

  C math, TR1, C++11, including all trig, pow, and special log1m, erf, erfc, fma, atan2, etc.

- Extensive library of **statistical functions**

  e.g., softmax, log gamma and digamma functions, beta functions, Bessel functions of first and second kind, etc.

- Efficient, arithmetically stable **compound functions**

  e.g., multiply log, log sum of exponentials, log inverse logit

Built-in Matrix Functions

- **Basic arithmetic**: all arithmetic operators
- **Elementwise arithmetic**: vectorized operations
- **Solvers**: matrix division, (log) determinant, inverse
- **Decompositions**: QR, Eigenvalues and Eigenvectors, Cholesky factorization, singular value decomposition
- **Compound Operations**: quadratic forms, variance scaling, etc.
- **Ordering, Slicing, Broadcasting**: sort, rank, block, rep
- **Reductions**: sum, product, norms
- **Specializations**: triangular, positive-definite,

Atomic Statements

- **Distribution**: `y ~ normal(mu,sigma)` (**increments log probability**)
- **Increment log density**: `target += lp;`
- **Assignment**: `y_hat = x * beta;`

Block Statements

- **Block**: `{ ...; ...; ...; }` (allows initial local variables)

Control Statements

- **For loop**: `for (n in 1:N) ...`
- **While loop**: `while (cond) ...`
- **Conditional**: `if (cond) ...; else if (cond) ...;  else ...;`
- **Break**: `break`
- **Continue**: `continue`

Statements with Side Effects

- **Print**: `print("theta=", theta);`
- **Reject**: `reject("arg to foo must be positive, found y=", y);`

Distribution Statement Increments Target

- A Stan program defines a log posterior
  - typically through log joint and Bayes's rule
- Sampling statements are just "syntactic sugar"
- A shorthand for incrementing the log posterior
- The following define the same* posterior
  - `y ~ poisson(lambda);`
  - `target += poisson_lupdf(y | lambda);`
- * up to a constant
- Sampling statement drops constant terms

Local Scope Blocks

```
y ~ bernoulli(theta);
```

is more efficient with sufficient statistics

```
{
  real sum_y;  // local variable
  sum_y = 0;
  for (n in 1:N)
    sum_y = sum_y + y[n];   // reassignment
  sum_y ~ binomial(N, theta);
}
```

- Simpler, but roughly same efficiency:

```
sum(y) ~ binomial(N, theta);
```

User-Defined Functions

- Declared and defined in **`functions`** block
- Invoked across program
- `functions` block example

```
functions {

  real relative_difference(real u, real v) {
    return 2 * fabs(u - v) / (fabs(u) + fabs(v));
  }

}
```

Special User-Defined Functions

- When declared with appropriate naming, user-defined functions may
  - be used in sampling statements: `real` return and suffix `_lpdf` or `_lpmf`
  - use RNGs: suffix `_rng`
  - use target accumulator: suffix `_lp`

User-Defined pdfs and cdfs

- May be used with sampling and PDF/PMF notation
  - **density**: suffix `_lpdf` if variate (first arg) is real
  - **mass**: Suffix `_lpmf` if variate (first arg) is integer

```
functions {
  real centered_normal_lpdf(real y, real sigma) {
    return -0.5 * (y / sigma)^2;
  }

model {
  y ~ centered_normal(2.5);                  // sampling

target += centered_normal_lpdf(y | 2.5);  // pdf notation
```

Target Increments

- May access target or use sampling statements
- Only usable in model block
  - must end in _lp

```
functions {
  vector non_center_lp(vector beta_std, real mu, real sigma) {
    beta_std ~ normal(0, 1);
    return mu + sigma * beta_raw;
  }
parameters {
  vector[K] beta_std;
model {
  vector[K] beta = non_center_lp(beta_std);
  y ~ normal(x * beta, sigma);
```

RNG Functions

- Only usable in transformed data and generated quantities blocks
  - must end in _rng

```
functions {
  real centered_normal_rng(real sigma) {
    return normal_rng(0, sigma);
  }
}
generated quantities {
  real alpha = centered_normal_rng(2.7);
```

A First Multilevel Model

Major League Baseball (MLB) Player Batting Ability

- Stan Case Study:

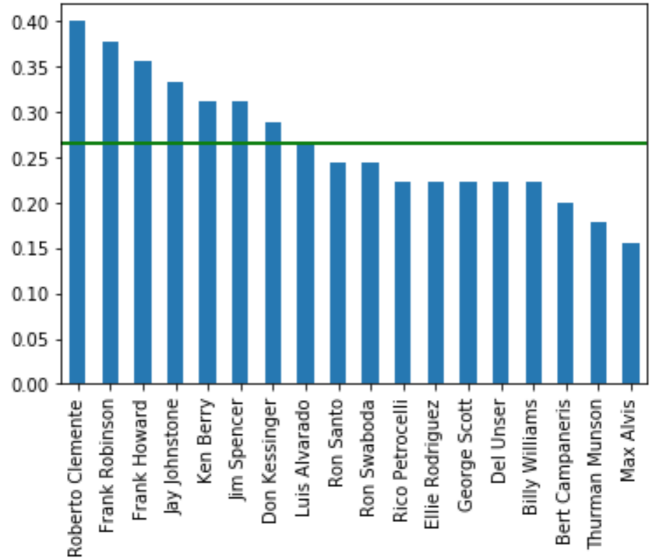
  Hierarchical Partial Pooling for Repeated Binary Trials

- Data: batting records from 1970 for 18 Major League Baseball players.
  - given number of hits in first 45 at-bats,
    estimate probability of a hit for a single at-bat.
    (roughly 3 at-bats per game, i.e., first 15 games of the season).
- Chance of getting a hit in a single at-bat: Bernoulli
- Chances of getting y number of hits in N at-bats: binomial
- Simple Beta-Binomial model, 2 flavors
  - all players are alike; all players are individuals
- Multilevel model
  - model both "average" player ability and individual player ability

Data: Batting Averages after 45 At-Bats

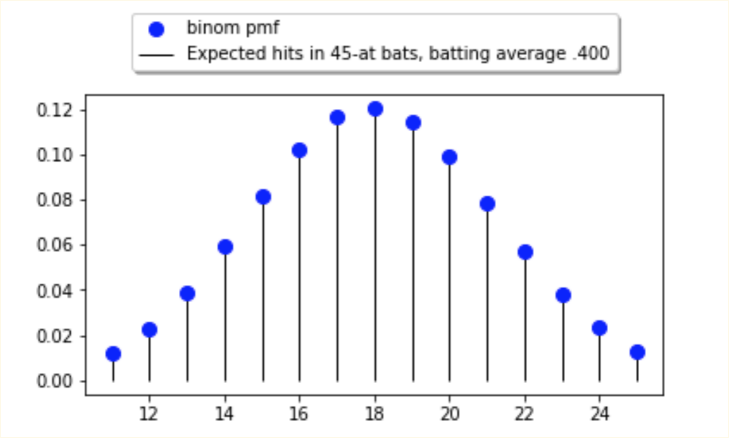
ID	Player	Hits	Average	SeasonAt-Bats	SeasonHits	SeasonAverage
1	Roberto Clemente	18	.400	412	145	.352
2	Frank Robinson	17	.378	471	144	.306
3	Frank Howard	16	.356	566	160	.283
4	Jay Johnstone	15	.333	320	76	.238
5	Ken Berry	14	.311	463	128	.276
6	Jim Spencer	14	.311	511	140	.274
7	Don Kessinger	13	.289	631	168	.266
8	Luis Alvarado	12	.267	183	41	.224
9	Ron Santo	11	.244	555	148	.267
10	Ron Swoboda	11	.244	245	57	.233
11	Rico Petrocelli	10	.222	583	152	.261
12	Ellie Rodríguez	10	.222	231	52	.225
13	George Scott	10	.222	480	142	.296
14	Del Unser	10	.222	322	83	.258
15	Walt Williams	10	.222	315	79	.251
16	Bert Campaneris	9	.200	603	168	.279
17	Thurman Munson	8	.178	453	137	.302
18	Max Alvis	7	.156	115	21	.183

Preliminary Data Analysis



- Green line pools data across all players
  - 810 at-bats
  - 215 hits
  - batting average: 265.4

Binomial Distribution: Roberto Clemente



With only 45 at-bats, the estimate of possible hits given 18 observed hits is very wide (uncertain).

Models 1, 2: Beta-Binomial Model

- Given the data, the model is the same Beta-Binomial model used by Laplace
  - The "complete-pooling" model considers that all players have the same ability
  - The "no-pooling" model computes the per-player ability

```
data {
  int<lower=0> N;  int<lower=0, upper=N> y;
}
parameters {
  real<lower=0, upper=1> theta;   // chance of getting a hit
}
model {
  theta ~ beta(1, 1);  y ~ binomial(N, theta);
}
generated quantities {
  real batting_average = 1000 * theta;
}
```

Live Demo: Run Model 1 in the Stan Playground.

- Posterior summary
  - R-hat, ESS are OK
  - average batting ability: 267
  - 90% central interval for complete pooling model: $241, 292$
- Model criticism:
  - Wide 90% central interval
  - We have 100+ season's of baseball data on over $10,000$ players

    "it is very unlikely that all players have the same chance of success."

Hierarchical Models

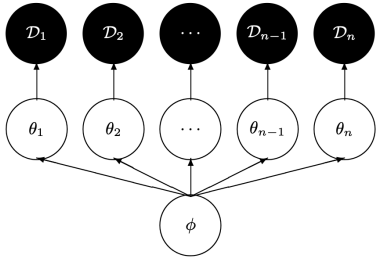


FIG. 1. In hierarchical models "local" parameters, θ, interact via a common dependency on "global" parameters, ϕ. The interactions allow the measured data, $\mathcal{D}$, to inform all of the θ instead of just their immediate parent. More general constructions repeat this structure, either over different sets of parameters or additional layers of hierarchy.

A simple hierarchical* model

- Data: pairs (value, group-id)
- Population-level parameter ϕ
- Local parameters $\theta_1, ...\theta_N$, 1 per group
- Local parameters draw common information from global parameter *partial pooling of information*
- All parameters estimated jointly

Hamiltonian Monte Carlo for Hierarchical Models, M. J. Betancourt and Mark Girolami

*See All the names for hierarchical and multilevel modeling

Model 3: Partial Pooling Model of MLB Player Batting Ability

- MLB Players belong to a population of players
- Estimate the population properties **jointly** with the player abilities
  - implicitly controls the amount of pooling
  - the more variable the population estimate, the less pooling
- Place a prior on the player ability
  - the player ability parameter is θ, the probability of getting a hit in an at-bat
  - θ is a probability (range $0, 1$)
  - the prior which matches this range is a **Beta distribution**
  - instead of a uniform prior, `beta(1, 1)`, **estimate** the parameters of the Beta distribution
- What Exactly Do We Want to Estimate?

Reparameterizing the Beta Distribution

- The Beta distribution can be specified by positive shape parameters α, β.
  - In the Beta-Binomial model, $\alpha - 1$ and $\beta - 1$ are interpreted as prior **pseudo-counts** of successes and failures
- An equivalent parameterization uses the **mean** ϕ and **concentration** κ:

$$\phi = \frac{\alpha}{\alpha + \beta}, \quad \kappa = \alpha + \beta \qquad \Longleftrightarrow \qquad \alpha = \phi\kappa, \quad \beta = (1 - \phi)\kappa$$

  - therefore,

$$\theta_i \sim \text{Beta}\left(\phi\kappa, (1 - \phi)\kappa\right)$$

- ϕ determines the **center** of the distribution:

$$\mathbb{E}[\theta_i \mid \phi, \kappa] = \phi$$

- κ determines how tightly values are concentrated around ϕ:

$$\text{Var}(\theta_i \mid \phi, \kappa) = \frac{\phi(1 - \phi)}{\kappa + 1}$$

Model 3: First Stan Hierarchical Model

```
data {
  int<lower=0> N; // players
  array[N] int<lower=0> K; // at-bats
  array[N] int<lower=0> y; // hits
}
parameters {
  real<lower=0, upper=1> phi; // population chance of success
  real<lower=1> kappa; // population concentration
  vector<lower=0, upper=1>[N] theta;  // batting ability
}
model {
  kappa ~ pareto(1, 1.5); // hyperprior
  theta ~ beta(phi * kappa, (1 - phi) * kappa);  // prior
  y ~ binomial(K, theta);
}
generated quantities {
  vector[N] batting_average = 1000 * theta;
}
```

Prior on the Population of Players

- Hierarchical model:

$$\theta \sim \text{Beta}(\phi\kappa, (1-\phi)\kappa)$$

- ϕ: population-average batting ability
- κ: **concentration** of player abilities around ϕ

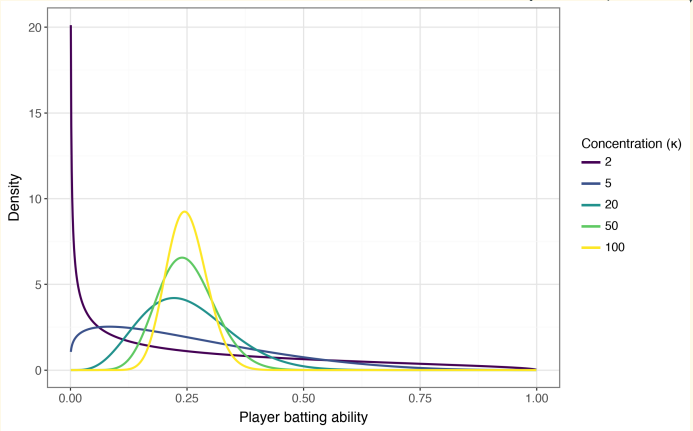
$$\text{Var}(\theta_i \mid \phi, \kappa) = \frac{\phi(1-\phi)}{\kappa+1}$$

- Larger $\kappa \rightarrow$ players are more similar
- Smaller $\kappa \rightarrow$ more variation among players
- We need a prior for κ

What Does Concentration Mean?

Fix the population mean at $\phi = 0.25$ and vary the concentration κ to see the resulting distribution of **player ability** θ:

$$\theta \sim \text{Beta}(0.25\kappa, 0.75\kappa)$$



- $\kappa = 2$: large variation
- $\kappa = 20$: moderate variation
- $\kappa = 100$: small variation
- Increasing κ concentrates player abilities around ϕ

Prior on the Population Concentration

- Parameters block declaration: `real<lower=1> kappa;`
- Model block prior: `kappa ~ pareto(1, 1.5);`
  - because κ is a parameter of the prior on θ, this is a **hyperprior**
- Pareto$(1, 1.5)$ has support $\kappa \geq 1$
  - $y_{\text{min}} = 1$: lower bound
  - $\alpha = 1.5$: shape parameter controlling the right tail
- Most prior probability is on smaller values of κ
  - favors more variation among player abilities
- Long right tail allows large values of κ
  - allows player abilities to be tightly concentrated around ϕ

Live Demo: Run Model 3 in the Stan Playground.

- The model can estimate the population mean `phi`
  - even though the default prior is uniform `phi ~ beta(1, 1);`
- The estimates of `theta` and `batting_ability` match the case study
- Stan has difficulty exploring the marginal posterior of `kappa`
  - `kappa` sometimes has R-hat > 1.01, ESS is always lower than other parameters.
  - the estimate of `kappa` is broad and strongly right-skewed (small population variation).

Quantile	κ
5%	20.33
50%	64.63
95%	404.5

Model 4: Partial Pooling on the Log-Odds Scale

- Instead of directly modeling the chance of success: $\theta_n \in (0, 1)$,

  Model 4 models the **log-odds** α_n:

$$\alpha_n = \text{logit}(\theta_n) = \log \frac{\theta_n}{1 - \theta_n}, \qquad \alpha_n \in (-\infty, \infty)$$

- On the unconstrained log-odds scale, player abilities α_n are given a Normal prior:

$$\alpha_n \sim \text{Normal}(\mu, \sigma)$$

- The likelihood is still binomial; the **inverse logit** transforms log-odds $\alpha_n \in (-\infty, \infty)$ back to a probability in $(0, 1)$:

$$y_n \sim \text{Binomial}\left(K_n, \text{logit}^{-1}(\alpha_n)\right)$$

- Stan combines the inverse-logit transformation and binomial likelihood:

```
y ~ binomial_logit(K, alpha);
```

Model 4: Likelihood and Priors

- The chance of success is reparameterized using Stan's `binomial_logit` function.

```
y ~ binomial_logit(K, alpha);
```

- `alpha` has a Normal **hierarchical** prior; it is distributed according to a population mean `mu` and scale `sigma`:

```
mu ~ normal(-1, 1); // hyperprior
sigma ~ std_normal(); // hyperprior
alpha ~ normal(mu, sigma); // prior (hierarchical)
```

- Population mean `mu` is specified on the **log-odds** scale, with location -1, scale 1. On the log-odds scale, $\text{logit}^{-1}(-1) \approx 0.27$.
- The chance of success θ, is recovered in the generated quantities block:

```
vector[N] theta = inv_logit(alpha);
```

Model 4: Direct Encoding

```
data {
  int<lower=0> N; array[N] int<lower=0> K;  array[N] int<lower=0> y;
}
parameters {
  real mu; // population mean of success log-odds
  real<lower=0> sigma; // population sd of success log-odds
  vector[N] alpha; // success log-odds
}
model {
  mu ~ normal(-1, 1); // hyperprior
  sigma ~ std_normal(); // hyperprior
  alpha ~ normal(mu, sigma); // prior (hierarchical)
  y ~ binomial_logit(K, alpha); // likelihood
}
generated quantities {
  vector[N] theta = inv_logit(alpha);
  vector[N] batting_average = 1000 * theta;
}
```

Live Demo: Run Model 4 in the Stan Playground.

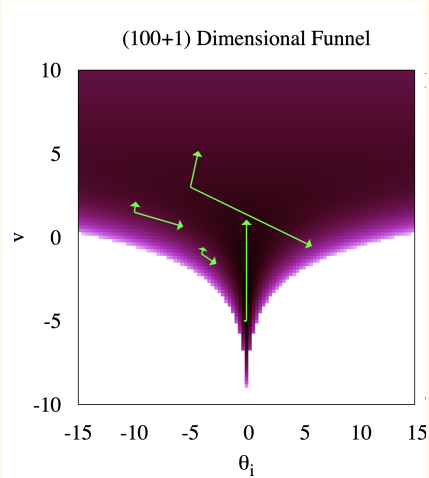
- The model can estimate the population mean `mu`
- The estimates of `theta` and `batting_ability` match the case study
- Stan has difficulty exploring the marginal posterior of `sigma`, the population scale.
  - `sigma` often has R-hat > 1.01, ESS is always lower than other parameters.
  - the estimate of `sigma` is broad and strongly right-skewed (small population variance).

Quantile	σ
5%	0.06
50%	0.18
95%	0.38

Direct Coupling of Hierarchical Parameters

- The local parameter `alpha`, (chance of success in an individual at bat), depends directly on the population-level parameter `sigma`.
- In 2-dimensions, a visualization of this dependency resembles a funnel shape - a narrow neck, a broad mouth.
- The hierarchical structure induces correlations *which change dramatically according to position*
- The HMC sampler uses a single global metric and step size
  - metric cannot describe curvature that changes across the posterior
  - in the neck of the funnel, step sizes shrink
  - the narrowest part of the funnel neck cannot be explored
  - small steps cannot explore the mouth of the funnel

Visualization of the Funnel



- The density of the hierarchical **prior**
  - x-axis: local parameter θ_i
  - y-axis: population scale parameter
- In a low-data regime, the posterior density resembles the prior density
  - HMC's global tuning parameters fail
- With more data, this pathology resolves itself

[(Figure 2, Betancourt and Girolami, 2013]((https://arxiv.org/abs/1312.0906)

Non-Centering: Sample on a Standard Scale

- The centered hierarchical prior is

$$\alpha_n \mid \mu, \sigma \sim \mathcal{N}(\mu, \sigma).$$

- The non-centered parameterization defines a standardized player effect:

$$z_n = \frac{\alpha_n - \mu}{\sigma}, \qquad z_n \sim \mathcal{N}(0, 1).$$

- Recover the player's log odds with an affine transformation:

$$\alpha_n = \mu + \sigma z_n.$$

- The sampler explores z, whose prior scale is always 1.
- The likelihood still uses the original player log odds:

  $y_n \sim \text{BinomialLogit}(K_n, \mu + \sigma z_n)$.
- Same statistical model; different coordinates for computation.

Model 4: Non-centered Parameterization

```
data {
  int<lower=0> N; array[N] int<lower=0> K; array[N] int<lower=0> y;
}
parameters {
  real mu;
  real<lower=0> sigma;
  vector[N] alpha_std;
}
model {
  mu ~ normal(-1, 1);
  sigma ~ std_normal();
  alpha_std ~ std_normal();
  y ~ binomial_logit(K, mu + sigma * alpha_std);
}
generated quantities {
  vector[N] theta = inv_logit(mu + sigma * alpha_std);
  vector[N] batting_average = 1000 * theta;
}
```

Live Demo: Fix Model 4 in the Stan Playground.

- Does this fix R-hat, ESS of `sigma`? *yes*
  - it improves computation
- Does this tighten the marginal posterior estimate of `sigma`? *no*
  - it cannot improve estimation - it supplies no new information